25-09

TESTING DE SOFTWARE

09 - Testing de software - Parte tradicional - Fundamentos

Funciones principales del testing

Verificar (Funciona correctamente? Tiene errores?)
Validar (El sistema hace lo que realmente se pide? Apunta a corrección)

Partiendo de la base que el PUD tiene workflows:

1. Requerimientos
2. Analisis
3. Diseno
4. Implementacion (TDD - Test Driven Development)
5. Prueba o Test (AQUI PARTICIPA EL TESTING)
6. Despliegue

Definición de Testing

ACTIVIDAD DE CONTROL DE CALIDAD, NO DE ASEGURAMIENTO CALIDAD

El testing no asegura calidad. No se puede asegurar que el sw no tiene defectos. El testing colabora dentro del QA (quality assurance - aseguramiento de calidad) , desde el control de la calidad (uno de los aspectos dentro del QA) porque la calidad es algo que tiene que estar embebido en el producto desde que iniciamos a trabajar, no se agrega al final. **El testing es una actividad cuyo objetivo es ENCONTRAR DEFECTOS**. Desde el contexto del proceso, el testing llega tarde, pone en evidencia lo que antes se hizo mal o que no se hicieron. Por eso el testing no asegura calidad. Un proceso destructivo de encontrar defectos **cuya presencia se asume (va a haber errores)** en el código. Un testing exitoso encuentra defectos. **Un desarrollo no exitoso lleva a un testing exitoso**.

Conceptos importantes

Error vs Defecto

No son sinónimos. Usamos el concepto de falla para diferenciarlos. Los diferencia el momento en el que se introduce y se lo detecta. Cuando algo no se traslada de la etapa en la que se introdujo: es un error. Cuando se introduce en una etapa y se detecta en una etapa posterior: es un defecto. Un error de requerimiento encontrado en una validación es un error y un error de arquitectura en una prueba también. Una falla es un mal funcionamiento del sistema, los errores y defectos los provocamos nosotros, *EL SISTEMA NO SE COMPORTA COMO SE DEBERÍA COMPORTAR.* Un defecto puede o no generar una falla en el sistema. La mayoría sí pero no todos. Cuando aparece un defecto aparece un problema y hay que resolverlo. Debemos clasificar los defectos por Severidad (*que tan grave es para el sistema en el funcionamiento ese defecto*) y por Prioridad (participa la perspectiva del usuario final o cliente, es del PO, del negocio y determina urgencia, yo necesito esto y que tan rapido lo necesito, cuanto puedo vivir sin esto):

Severidad	Prioridad
1 – Bloqueante	1 – Urgencia
2 – Crítico	2 – Alta
3 – Mayor	3 – Media
4 – Menor	4 – Baja
5 - Cosmético	

Dentro de severidad (gravedad en el software)

- Bloqueante (el sistema no anda)
- Critico (el sistema anda)
- Mayor (te permite un camino alternativo para resolver la funcionalidad que no pudiste resolver, por eso no es critico, HAY UNA ALTERNATIVA O PLAN B)
- Menor (te deja, podes seguir operando igual pero sigue siendo una falla)

Niveles de prueba

Tienen que ver con en donde estamos mirando. Cuando desarrollamos, lo hacemos de lo general a lo particular. Cuando se testea se hace lo contrario, se prueba de lo más chico a lo más grande. Lo primero que hacer es probar a nivel Componentes (Nivel de prueba unitaria o primer nivel de testing)

Pruebas unitarias (implementacion) **desarrollador**

Primer nivel de testing. Se hace dentro Implementacion (PUD). Pruebas a nivel Componente. Busca proveer un mecanismo de verificacion de la comprension del requerimiento/funcionalidad que estoy implementando.

Pruebas de integración (implementacion) **tester**

Segundo nivel de testing. Prueba la union de los componentes. Se suele llamar “Prueba de interfaces”. Los componentes se comunican entre si como se deben comunicar y el que pide algo reciba lo que se necesita y que el otro de lo que se supone que debe dar. El resultado es obtener un “Build” –> Versión del software que integra los componentes que queremos mandar a testing para ser probados. La teoria dice que se deberia hacer en Prueba (PUD), en la practica, en la mayoria de los casos, se hace en el ambito de desarrollo/Implementacion (PUD) por un cargo superior al del tester.
En este tipo de pruebas buscamos encontrar defectos ya que esta orientado a la prueba de interfaces (boundaries/fronteras entre los diferentes componentes) “Probar las **integraciones** entre los diferentes componentes”

Pruebas de sistema **tester**

Correspondiente a pruebas de iteraciones. Se prueba sobre el BUILD (conjunto de componentes integrados que resuelven un conjunto de requerimientos)

Pruebas de aceptación (pre-produccion) **usuario final o po**

QAT (User aceptance test). Pruebas de aceptacion de usuario. Las pruebas las hace el usuario final. Se hacen dentro de Despliegue (PUD)

Ambientes o entorno de trabajo

Espacio que se genera para poder hacer lo que se tiene que hacer con un SW:

- Desarrollar el sw
- Probar sw
- Usar sw

Entorno

Conjunto de herramientas que se necesitan disponibles desde el SO hasta librerías y versiones, sobre un Hardware con todas las versiones de todas las cosas, para que el sistema se pueda construir, probar o usar. Cada tarea tiene un entorno distinto. (diferentes monitores, etc)

Casos de prueba

Conjunto de instrucciones que describen paso a paso un escenario de uso de una funcionalidad del sistema. Es el artefacto más importante que tiene Testing. Cuando vamos a probar un SW necesitamos hacer un SW sistemático y que permita reproducir efectos. Si probamos sin casos de prueba, si aparece un defecto no lo vamos a poder reproducir (reportarlo o mostrar el paso a paso que provocó el defecto). Un defecto que no se puede reproducir no es un defecto. La actividad más cara de todas es el Testing. PAra disenarlos, sacamos la informacion de los requerimientos.

Estrategias de prueba

Caja blanca: mirás el código. Puedo ver loops infinitos, ramas de código inutiles, codigo malicioso. Estático. Busco abarcar la mayor cantidad de lineas de codigo en cada prueba.

Caja negra: no puedo ver el proceso, solo entradas y salidas. Si la salida es buena respecto a la entrada, bien. Es dinámico.

Cada uno es bueno en lo que no otro hace.

Caja blanca: métodos de cobertura. Cuántas lineas de codigo se analizarán. Con distintas tecnicas probamos los diferentes caminos

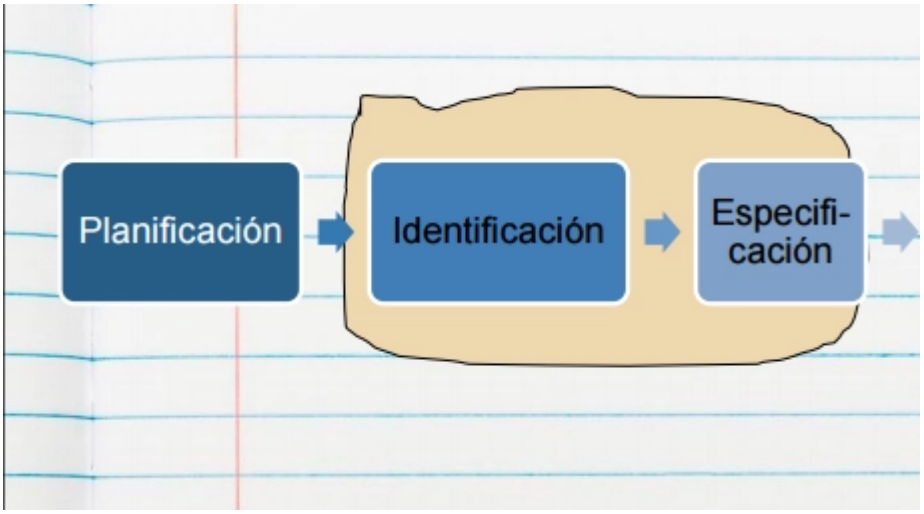
Metodos de prueba

—----------

Proceso de prueba



Planificación: calidad, riesgos. se planifica cuando se planifica el proyecto (planes de soporte), tengo que saber las estrategias, métodos, como obtener los datos. Plan de Prueba es el artefacto de salida

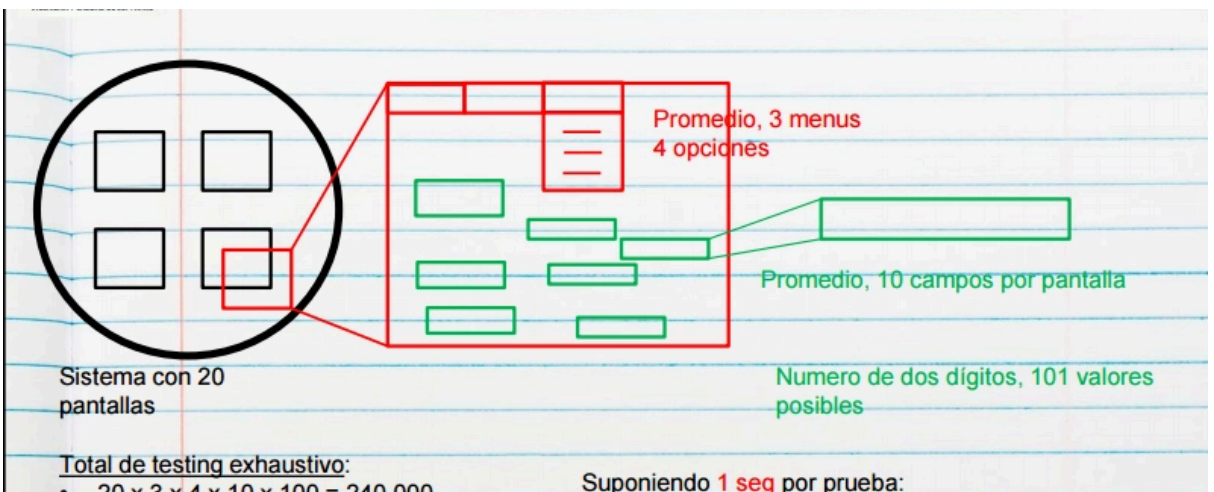


Identificación y especificacion: llamada diseño de pruebas. Preparar datos, armar las bd, configurar entornos. Hasta ahora 0 código

Ejecución: ejecutamos pruebas con implementación entrando. Salida: reporte de defectos, con severidad. Los defectos tienen que ser reproducibles, sino no se corrigen.

En el plan se acuerda cuando se da el fin de pruebas y debe ser conocido x todos, no siempre cuando ya no quedan defectos: es imposible por tema de plata y tiempo. Se termina de probar cuando hay 0 errores bloqueantes, críticos (ejemplo).

Testing exhaustivo es imposible:



Situación: probar RF y RNF. Testing Funcional y No Funcional: performance, hardware. Probar concurrencia, etc.

Las pruebas más dificiles son las NF.

Tipos de prueba

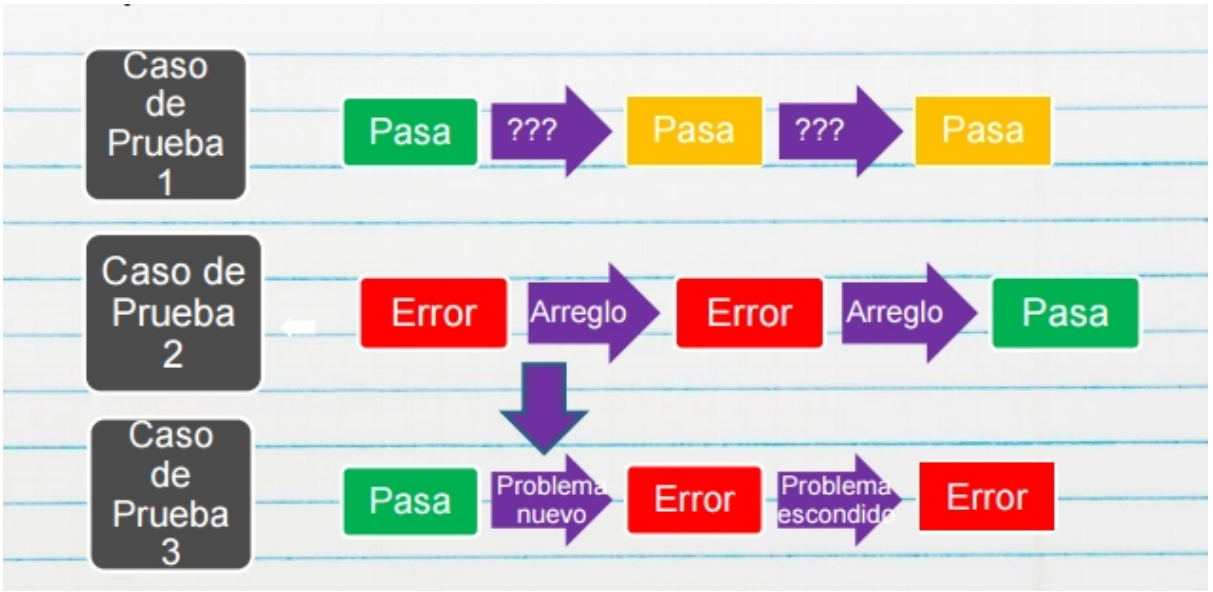
—-

Ciclos de prueba

Testing crea un conjunto de CdP en base a una nueva versión (5.2). Entra la versión y hacen los caso de pruebas y terminan. Se termina un ciclo y se hace un reporte de defectos con su severidad. Desarrollo lo recibe, negocia las correcciones y manda una nueva versión (5.3). Testing capaz tiene que ajustar los casos de prueba si son errores de requerimientos, pero si son de código no. Se vuelven a ejecutar ciclos de prueba hasta que se termine de verificar y validar la versión entregada.

Sin regresión

No se prueban todos los casos de prueba por cada vez que vuelve a entrar una versión mandada a corregir. Solo se controla que se hayan corregido los defectos reportados. El problema es que por cada error corregido, se introducen 3, entonces no sabemos si lo que pasó antes ahora pasa.



Con regresión

Se prueba todo de nuevo siempre aunq hayan pasado los anteriores casos de prueba en un ciclo anterior. Caro, pero se puede resolver con automatización

